

Relatório 7 - Prática de estado global e integração com API

Kaíque Medeiros Lima

1. Descrição da atividade

a. Estado local, global e de servidor

A atividade trabalha a diferença entre informações que pertencem a um componente, dados compartilhados pela aplicação e conteúdo que vem de um servidor. O projeto usa Next.js e TypeScript, com Zustand para o estado global, TanStack Query para os dados de servidor, Axios para as chamadas HTTP e Zod para validar os dados em tempo de execução. Cada ferramenta entra em uma parte do fluxo.

No dashboard, a busca por título e o filtro entre todos, lidos e não lidos ficam em `useState`, porque só essa tela precisa deles. O usuário logado e a marcação de posts lidos e favoritos ficam em stores do Zustand, porque são compartilhados e precisam sobreviver a uma recarga da página. Já os posts são dados externos, que podem estar carregando, falhar ou precisar de uma nova busca, e esse controle fica com o TanStack Query, sem montar manualmente todos esses estados com `useState`.

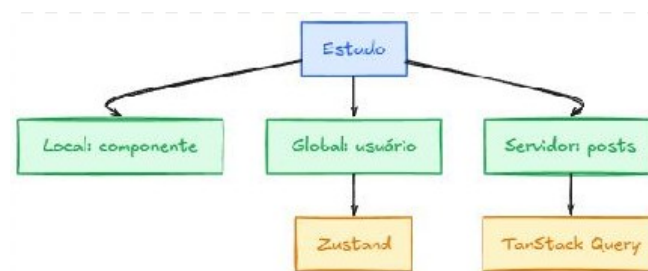


Figura 1 - Diferenças entre estado local, global e de servidor. Autoria própria, Excalidraw.

b. Stores do Zustand

A store `useUserStore` guarda o usuário com nome e e-mail, tipado com `LoggedUser`, o mesmo tipo inferido do schema de login, e oferece as ações `setUser` e `logout`. Ela usa o middleware `persist`, que salva o usuário no `localStorage`, então a sessão continua depois de recarregar a página. A opção `skipHydration` evita diferença entre o HTML gerado no servidor e o do navegador, e a reidratação é feita em um `useEffect` do cabeçalho. O `logout` limpa o usuário e também remove o token salvo.

A segunda store, `usePostsStore`, guarda os ids dos posts lidos e favoritos, com as ações `toggleRead` e `toggleFavorite`. Ela também é persistida, então as marcações continuam após recarregar a página. O dashboard lê essas listas para mudar o visual de cada item, contar quantos posts foram lidos e aplicar o filtro. Assim, o estado global é de fato alterado e consumido pela interface, e não fica apenas com um valor inicial.

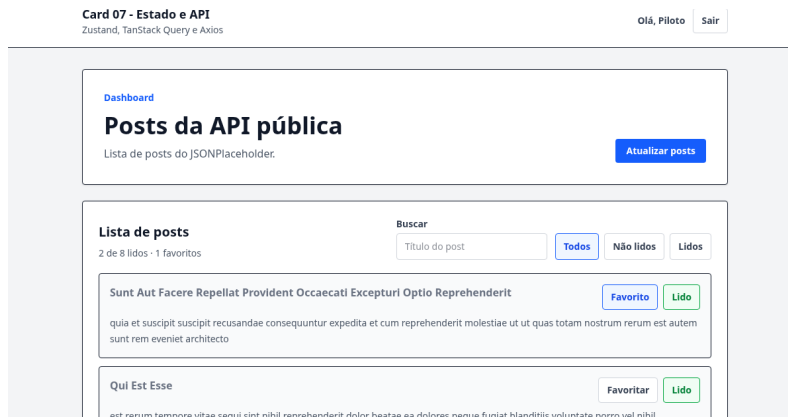


Figura 2 - Dashboard com usuário logado, um post favorito e dois lidos. Autoria própria, navegador.



Figura 3 - Filtro de posts lidos aplicado com estado local sobre a store. Autoria própria, navegador.

c. Cliente Axios e interceptors

O cliente HTTP fica em `data/services/api` e usa `NEXT_PUBLIC_API_URL` como endereço base, definido no `.env.example`. O código não tem um endereço padrão: se a variável faltar, o interceptor de requisição recusa a chamada com uma mensagem que aponta a variável ausente, em vez de mandar a requisição para um endereço escondido no código. O mesmo interceptor de requisição lê o token do `localStorage` e, quando ele existe, acrescenta o cabeçalho `Authorization` no formato `Bearer`. Isso concentra a preparação das chamadas em um só lugar, sem repetir o cabeçalho em cada serviço.

Um interceptor de resposta trata o status 401. Quando a API responde que a sessão não é válida, ele chama o `logout` da store, que apaga o token e o usuário ao mesmo tempo. Com isso, o ciclo do token fica fechado: ele é gravado no login, enviado em cada requisição e removido no logout ou quando o servidor recusa a sessão.

d. Login com `useMutation`

O botão de entrada do cabeçalho chama o hook `useLoginMutation`, que usa o `useMutation` do TanStack Query para executar o `AuthService.login` e, em caso de sucesso, salvar o usuário na store. O serviço valida as credenciais com o `loginSchema` do Zod antes de qualquer chamada, envia o POST para `/auth/login` e valida a resposta com o `loginResponseSchema`, que exige um token e um usuário. Só então o token é gravado.

O JSONPlaceholder não possui uma rota de login e responde 404. Por isso, com a variável `NEXT_PUBLIC_USE MOCK_AUTH` ligada, apenas esse 404 da rota `/auth/login` é convertido em uma sessão simulada, com um token gerado no navegador. Erros de rede, timeout, 500 ou 404 de outra rota continuam sendo lançados e aparecem na tela como falha de login. A entrada é uma simulação para a demonstração, e a senha não é conferida por nenhum servidor, mas o fluxo de estado e de token é o mesmo de uma API real.

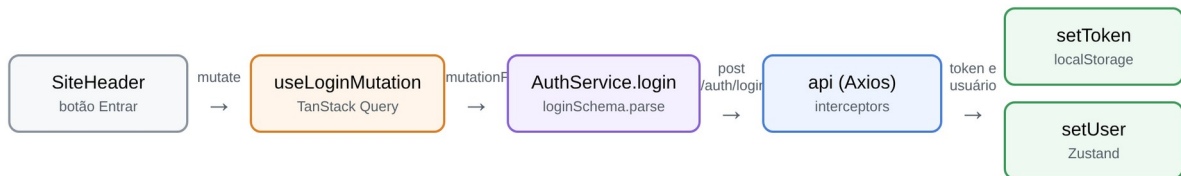


Figura 4 - Fluxo do login, do botão até o token e a store. Autoria própria.

```
src/modules/auth/data/services/auth/index.ts

1 static async login(payload: unknown): Promise<LoggedUser> {
2   const data = loginSchema.parse(payload);
3
4   try {
5     const response = await api.post<unknown>("/auth/login", data);
6     const { token, user } = loginResponseSchema.parse(response.data);
7     setToken(token);
8     return user;
9   } catch (error) {
10    if (!isMockEligibleError(error)) throw error;
11    const user = { name: nameFromEmail(data.email), email: data.email };
12    setToken(createMockToken());
13    return user;
14  }
15 }
```

Figura 5 - Login com validação da entrada e da resposta e simulação restrita ao 404. Código da branch do Card 07.

e. Serviço de posts e validação em runtime

O `PostsService` busca `/posts?_limit=8` pelo cliente `Axios`. O retorno é tratado como `unknown` e passa pelo `postsSchema` do `Zod` antes de ser devolvido. O tipo `Post` é gerado com `z.infer` a partir do mesmo schema, então o contrato de tipo e a validação ficam em um único lugar.

Tipar a chamada apenas com o genérico do `Axios` faria o `TypeScript` confiar no formato da resposta sem conferir nada durante a execução. Com o `parse`, se a API mudar ou devolver um campo inválido, o erro aparece no serviço, perto da origem, e a consulta vai para o estado de erro, em vez de quebrar mais tarde na renderização.

```
src/modules/posts/data/services/posts/index.ts

1 export class PostsService {
2   static async getAll(): Promise<Post[]> {
3     const response = await api.get<unknown>("/posts?_limit=8");
4
5     // valida o payload em runtime antes de entregar os dados tipados
6     return postsSchema.parse(response.data);
7   }
8 }
```

Figura 6 - Serviço de posts validando a resposta com Zod. Código da branch do Card 07.

f. Consulta, carregamento e erro no dashboard

O hook `usePostsQuery` liga a `queryKey` `["posts"]` ao `PostsService.getAll`, e o `ReactQueryProvider` envolve a aplicação no layout principal, criando o `QueryClient` uma única vez com a forma inicializadora do `useState`. O `QueryClient` recebe um `staleTime` padrão de cinco minutos, porque os posts mudam raramente e não faz sentido buscar tudo de novo a cada troca de foco da janela ou remontagem da tela. O dashboard recebe os posts junto com `isPending`, `isError` e `isFetching` e mostra uma mensagem de carregamento, uma mensagem de erro ou a lista, usando o id do post como chave.

O botão `Atualizar posts` chama `refetch` e fica desabilitado enquanto `isFetching` estiver ativo, com o texto `Atualizando`. Quando nenhum post corresponde à busca ou ao filtro, aparece uma mensagem de lista vazia. Para registrar o estado de erro, a rota de posts foi forçada a responder 500 no navegador, e a tela mostrou o aviso sem quebrar o restante da página.

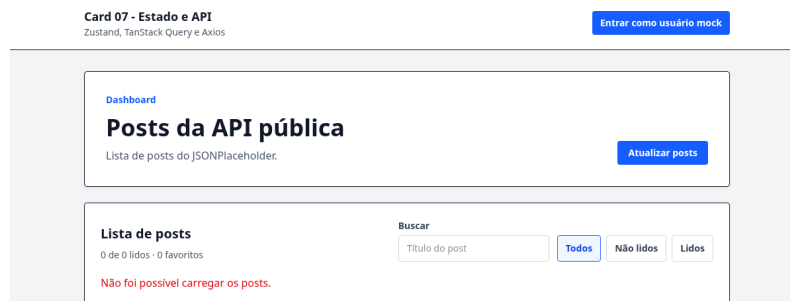


Figura 7 - Estado de erro quando a API de posts falha. Autoria própria, navegador.

g. Testes automatizados

O projeto tem 25 testes com Vitest, distribuídos em 7 arquivos. Eles cobrem as duas stores, a leitura e remoção do token, o logout disparado pelo 401 no interceptor, a falha quando `NEXT_PUBLIC_API_URL` não está definida, os schemas de login e de posts e o `AuthService`, verificando a resposta válida, a simulação apenas no 404 de `/auth/login` e a propagação dos demais erros. Todos os testes passam.

A formatação do código é feita pelo Prettier, com os scripts `format` e `format:check`, e o ESLint cuida do lint. Assim o estilo não depende de cada editor, e a verificação pode ser feita com um único comando antes do commit.

```
$ pnpm test
RUN v5.0.1 ~/CARD 07/estado-api
✓ user/index.test.ts (3 tests)
✓ posts/index.test.ts (2 tests)
✓ api/token.test.ts (3 tests)
✓ api/index.test.ts (2 tests)
✓ schemas/post.test.ts (3 tests)
✓ schemas/login.test.ts (7 tests)
✓ services/auth/index.test.ts (5 tests)

Test Files  7 passed (7)
Tests      25 passed (25)
```

Figura 8 - Execução dos testes do projeto. Autoria própria, terminal.

2. Conclusões

O card ajuda a entender por que nem todo estado precisa ser tratado da mesma forma. O `useState` atende ao que é da tela, o `Zustand` guarda o que é compartilhado e precisa persistir, e o `TanStack Query` cuida dos dados de servidor, com carregamento, erro e nova busca. O `Axios` concentra a configuração das chamadas e o ciclo do token nos interceptors, e o `Zod` garante que os dados externos tenham o formato esperado antes de chegar à interface. O limite do projeto é o login simulado, já que a API pública não tem autenticação, mas a estrutura já está pronta para trocar a simulação por um backend real.

3. Referências

<https://zustand.docs.pmnd.rs/>

<https://zustand.docs.pmnd.rs/integrations/persisting-store-data>

<https://tanstack.com/query/latest/docs/framework/react/overview>

<https://axios.rest/pages/getting-started/first-steps>

<https://axios-http.com/docs/interceptors>

<https://zod.dev/>

<https://prettier.io/docs/>